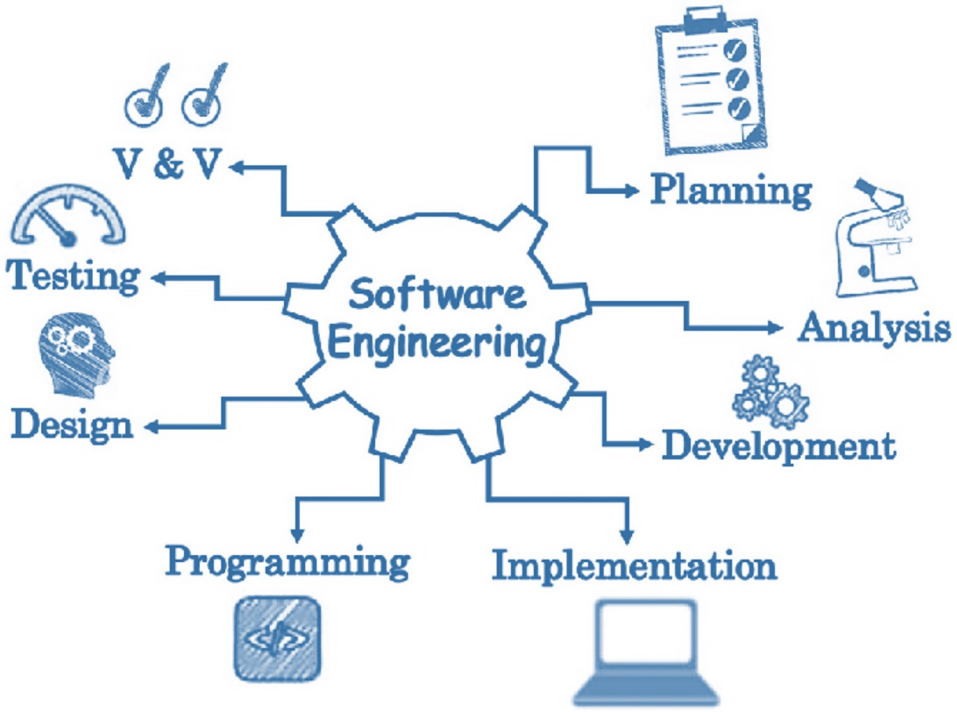
Software Engineering (CMP 519.3)



- Instructor
- Birodh Rijal

birodh.rijal@eemc.edu.np

Course Objective

- Understand and apply the fundamental principles of software engineering and Unified Modeling Language (UML) basics, in the development of software solutions.
- Analyze and specify software requirements, develop use cases and scenarios, apply in object oriented software engineering
- Analyze and Evaluate Software Maintenance and Evolution Strategies
- Apply Advanced Object-Oriented Software Engineering

Books

Prescribed Books and References
Text Books: 1. Software Engineering: A Practitioner's Approach, "Roger S. Pressman 2. Bernd Bruegge & Allen H. Dutoit, Object-Oriented Software Engineering Using UML, Patterns, and Java™
References: 1. Software Engineering, " Ian Sommerville", ninth edition 2. Software Engineering Fundamentals, " Ali Behforooz and Frederick J. Hudson

Evaluation

Internal Evaluation	Weight	Marks	External Evaluation	Marks
Attendance & Class Participation	10%	60	Semester-End examination	40
Assignments (Presentation and Report)	20%			
Internal Assessment	40%			
Research/Project/Case Study Work	30%			
Total Internal		60	Total External	40
Full Marks: 60 + 40 = 100				

- An individual case study will be given to each students on a software project and will be analyzed with any UML CASE tool (like Rational Rose Enterprise Suit 2000) and implemented in OO.
- 30% of sessional marks will be allocated for evaluation.

Assignment

- Assignment will be done in groups
- Reports will be Presented by each of the groups
- The Assignment will cover various topics in the syllabus
- You have to submit the assignment report in **pdf format** through email.
- Late submission will **not** be considered for marking.
- Assignment will carry **20% (12 Marks)**of your internal marks.

Chapter 1: Introduction

Software Engineering

- The seminal definition:
  - *[Software engineering is] the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.*
- The IEEE definition:
  - *Software Engineering:*
  - *The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.*
  - *"And yet, a "systematic, disciplined, and quantifiable" approach applied by one software team may be burdensome to another. We need discipline, but we also need adaptability and agility."*

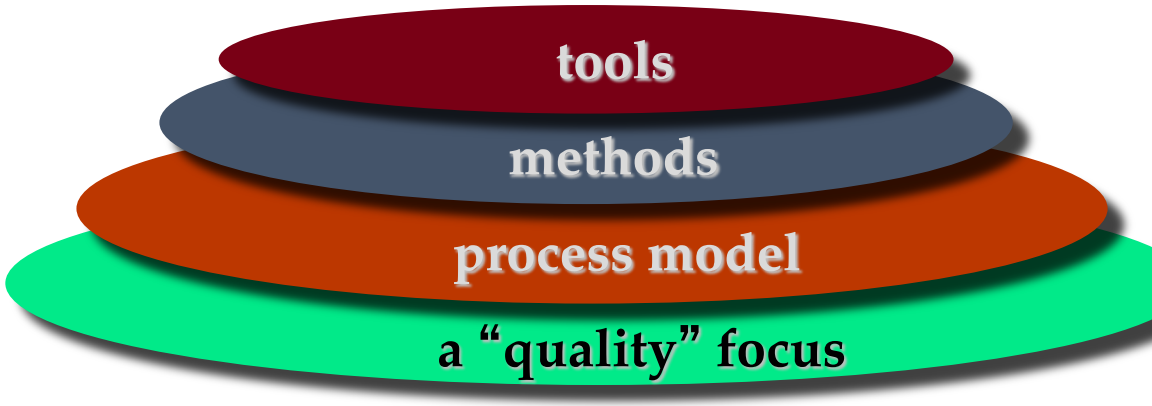
  - *Excerpt From: PRESSMAN;. "ISE EBook Online Access for Software Engineering: A Practitioner's Approach." Apple Books.*

Software Categories

- System software: such as compilers, editors, file management utilities
- Application software**:** stand-alone programs for specific needs.
- Engineering/scientific software: Characterized by “number crunching”algorithms. such as automotive stress analysis, molecular biology, orbital dynamics etc
- Embedded software resides within a product or system. (key pad control of a microwave oven, digital function of dashboard display in a car)
- Product-line software focus on a limited marketplace to address mass consumer market. (word processing, graphics, database management)
- WebApps (Web applications) network centric software. As web 2.0 emerges, more sophisticated computing environments is supported integrated with remote database and business applications.
- Mobile Apps: Runs on mobile device
- AI software uses non-numerical algorithm to solve complex problem. Robotics, expert system, pattern recognition game playing

Software Engineering

- Any engineering approach must rest on organizational commitment to **quality** which fosters a continuous process improvement culture.
- **Process** layer as the foundation defines a framework with activities for effective delivery of software engineering technology. Establish the context where products (model, data, report, and forms) are produced, milestone are established, quality is ensured and change is managed.
- **Method** provides technical how-to's for building software. It encompasses many tasks including communication, requirement analysis, design modeling, program construction, testing and support.
- **Tools** provide automated or semi-automated support for the process and methods.



Software Engineering Concept

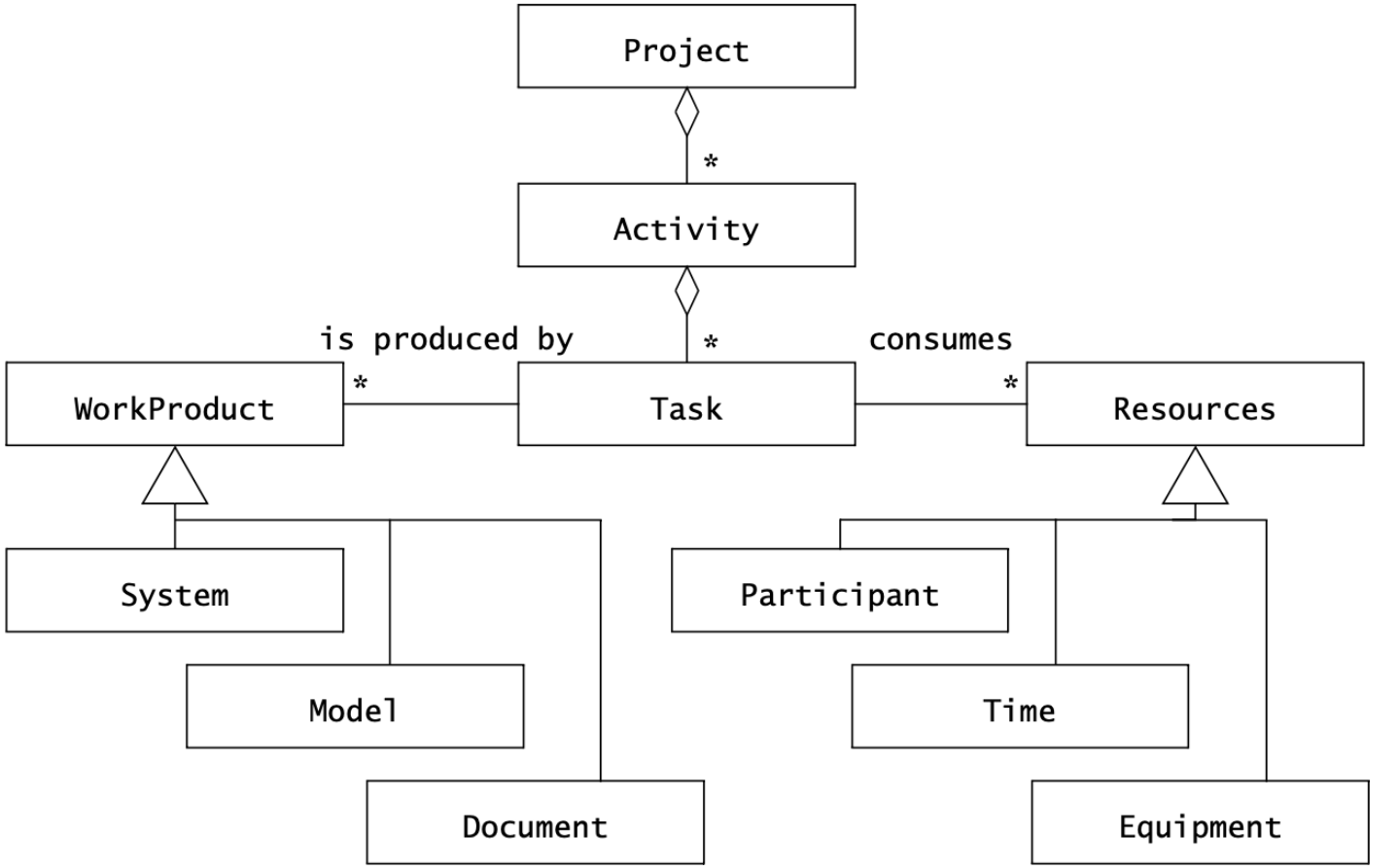


Figure 1-1 Software engineering concepts depicted as a UML class diagram [OMG, 2009].

Participant and Role

- We refer to all the persons involved in the project as **participants**.
- We refer to a set of responsibilities in the project or the system as a **role**.
- A role is associated with a set of **tasks** and is assigned to a participant. The same participant can fill multiple roles.

> `TicketDistributor` is a machine that distributes tickets for trains. Travelers have the option of selecting a ticket for a single trip or for multiple trips, or selecting a time card for a day or a week. The `TicketDistributor` computes the price of the requested ticket based on the area in which the trip will take place and whether the traveler is a child or an adult. The `TicketDistributor` must be able to handle several exceptions, such as travelers who do not complete the transaction, travelers who attempt to pay with large bills, and resource outages, such as running out of tickets, change, or power.

Participant and Role Software Projects

- We refer to all the persons involved in the project as **participants**.
- We refer to a set of responsibilities in the project or the system as a **role**.
- A role is associated with a set of **tasks** and is assigned to a participant. The same participant can fill multiple roles.

> `TicketDistributor` is a machine that distributes tickets for trains. Travelers have the option of selecting a ticket for a single trip or for multiple trips, or selecting a time card for a day or a week. The `TicketDistributor` computes the price of the requested ticket based on the area in which the trip will take place and whether the traveler is a child or an adult. The `TicketDistributor` must be able to handle several exceptions, such as travelers who do not complete the transaction, travelers who attempt to pay with large bills, and resource outages, such as running out of tickets, change, or power.

Participant and Role

Table 1-1 Examples of roles in software engineering for the `TicketDistributor` project.

Role	Responsibilities	Examples
Client	The client is responsible for providing the high-level requirements on the system and for defining the scope of the project (delivery date, budget, quality criteria).	Train company that contracts the `TicketDistributor`.
User	The user is responsible for providing domain knowledge about current user tasks. Note that the client and the user are usually filled by different persons.	Travelers
Manager	A manager is responsible for the work organization. This includes hiring staff, assigning them tasks, monitoring their progress, providing for their training, and generally managing the resources provided by the client for a successful delivery.	Alice (boss)
Human Factors Specialist	A human factors specialist is responsible for the usability of the system.	Zoe (Human Computer Interaction specialist)
Developer	A developer is responsible for the construction of the system, including specification, design, implementation, and testing. In large projects, the developer role is further specialized.	John (analyst), Marc (programmer), & Zoe (tester)[a]
Technical Writer	The technical writer is responsible for the documentation delivered to the client. A technical writer interviews developers, managers, and users to understand the system.	John

a. As `TicketDistributor` is a small project, Zoe fills both the human factor specialist and the tester roles, and John fills the analyst and the technical writer roles.

System and Models

- We use the term **system** as a collection of interconnected parts.
- Modeling is a way to deal with complexity by ignoring irrelevant details.
- We use the term **model** to refer to any abstraction of the system.
- *A TicketDistributor for an underground train is a system. Blueprints for the TicketDistributor, schematics of its electrical wiring, and object models of its software are models of the TicketDistributor. Note that a development project is itself a system that can be modeled. The project schedule, its budget, and its planned deadlines are models of the development project.*

System and Models

- Software systems are often represented using various models that help visualize, analyze, and design complex systems.
- **System**: A collection of components organized to achieve specific goals.
  Examples: **ERP System, Train reservation system**.
- **Models**:
  - **Structural Models**: Represent system components and their relationships (e.g., class diagrams).
  - **Behavioral Models**: Depict the dynamic behavior of the system (e.g., use-case and sequence diagrams).
  - **Architectural Models**: Show the system's high-level structure and interactions.

Work Products

- A **work product** is an artifact that is produced during the development, such as a document or a piece of software for other developers or for the client.
- We refer to a work product for the project's internal consumption as an **internal work product**.
- We refer to a work product that must be delivered to a client as a **deliverable**.
- Deliverables are generally defined prior to the start of the project and specified by a contract binding the developers with the client.

Work product	Type	Description
Specification	**Deliverable**	The specification describes the system from the user's point of view. It is used as a contractual document between the project and the client. The `TicketDistributor` specification describes in detail how the system should appear to the traveler.
Operation manual	**Deliverable**	The operation manual for the `TicketDistributor` is used by the staff of the train company responsible for installing and configuring the `TicketDistributor`. Such a manual describes, for example, how to change the price of tickets and the structure of the network into zones.
Status report	**Internal work product**	A status report describes at a given time the tasks that have been completed and the tasks that are still in progress. The status report is produced for the manager, Alice, and is usually not seen by the train company.
Test manual	**Internal work product**	The test plans and results are produced by the tester, Zoe. These documents track the known defects in the prototype `TicketDistributor` and their state of repair. These documents are usually not shared with the client.

Activities, Tasks, and Resources

- An **activity** is a set of tasks that is performed toward a specific purpose. For example, requirements elicitation is an activity whose purpose is to define with the client what the system will do.
- A **task** represents an atomic unit of work that can be managed: A manager assigns it to a developer, the developer carries it out, and the manager monitors the progress and completion of the task. Tasks consume resources, result in work products, and depend on work products produced by other tasks.
- **Resources** are assets that are used to accomplish work. Resources include time, equipment, and labor. When planning a project, a manager breaks down the work into tasks and assigns them to resources.

Table 1-3 Examples of activities, tasks, and resources for the `TicketDistributor` project.

Example	Type	Description
Requirements elicitation	**Activity**	The requirements elicitation activity includes obtaining and validating requirements and domain knowledge from the client and the users. The requirements elicitation activity produces the specification work product (Table 1-2).
Develop "Out of Change" test case for `TicketDistributor`	**Task**	This task, assigned to Zoe (the tester) focuses on verifying the behavior of the ticket distributor when it runs out of money and cannot give the correct change back to the user. This activity includes specifying the environment of the test, the sequence of inputs to be entered, and the expected outputs.
Review "Access Online Help" use case for usability	**Task**	This task, assigned to John (the human factors specialist) focuses on detecting usability issues in accessing the online help features of the system.
Tariff Database	**Resource**	The tariff database includes an example of tariff structure with a train network plan. This example is a resource provided by the client for requirements and testing.

Notations, Methods, and Methodologies

- A **notation** is a graphical or textual set of rules for representing a model. The Roman alphabet is a notation for representing words. UML (Unified Modeling Language [OMG, 2009]), the notation we use throughout this book, is a notation for representing object-oriented models.
- A **method** is a repeatable technique that specifies the steps involved in solving a specific problem. A sorting algorithm is a method for ordering elements of a list. Configuration management is a method for tracking change.
- A **methodology** is a collection of methods for solving a class of problems and specifies how and when each method should be used. Royce's methodology [Royce, 1998], the Object Modeling Technique (OMT [Rumbaugh et al., 1991]), the Booch methodology [Booch, 1994], and Catalysis [D'Souza & Wills, 1999] are object-oriented methodologies for developing software.

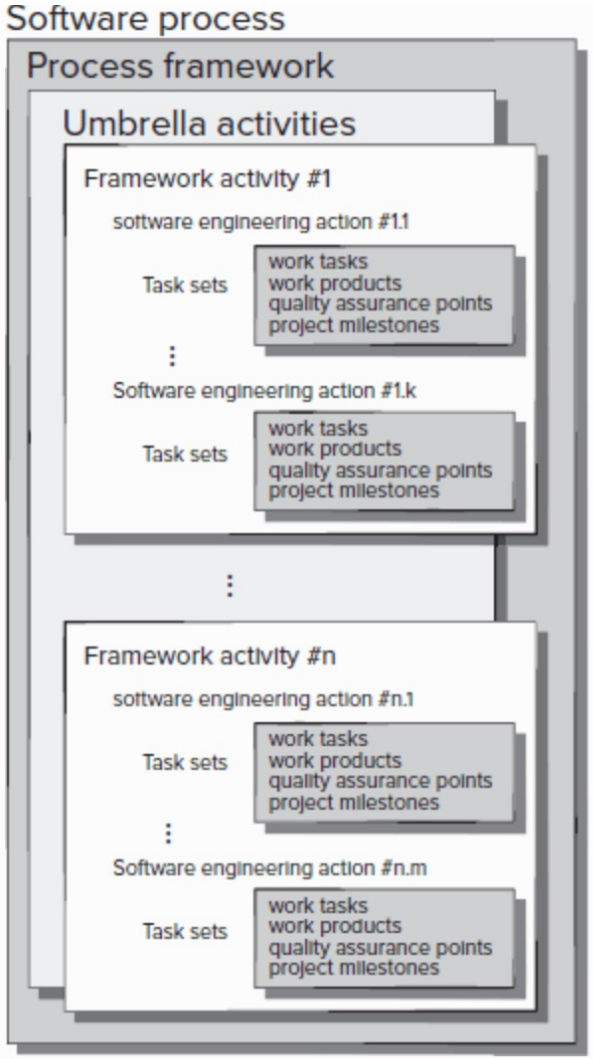
Overview of Software Process Model

Software Process Model

- We define a software process as a framework for the activities, actions, and tasks required to build high-quality software.
- Is “process” synonymous with “software engineering”?
- Answer could be yes or no.
- “software engineering is performed by creative, knowledgeable people who should adapt a mature software process so that it is appropriate for the products that they build and the demands of their marketplace.
- These models guide the teams to plan, manage, organize, optimize the resources, and ensure quality throughout the software development lifecycle

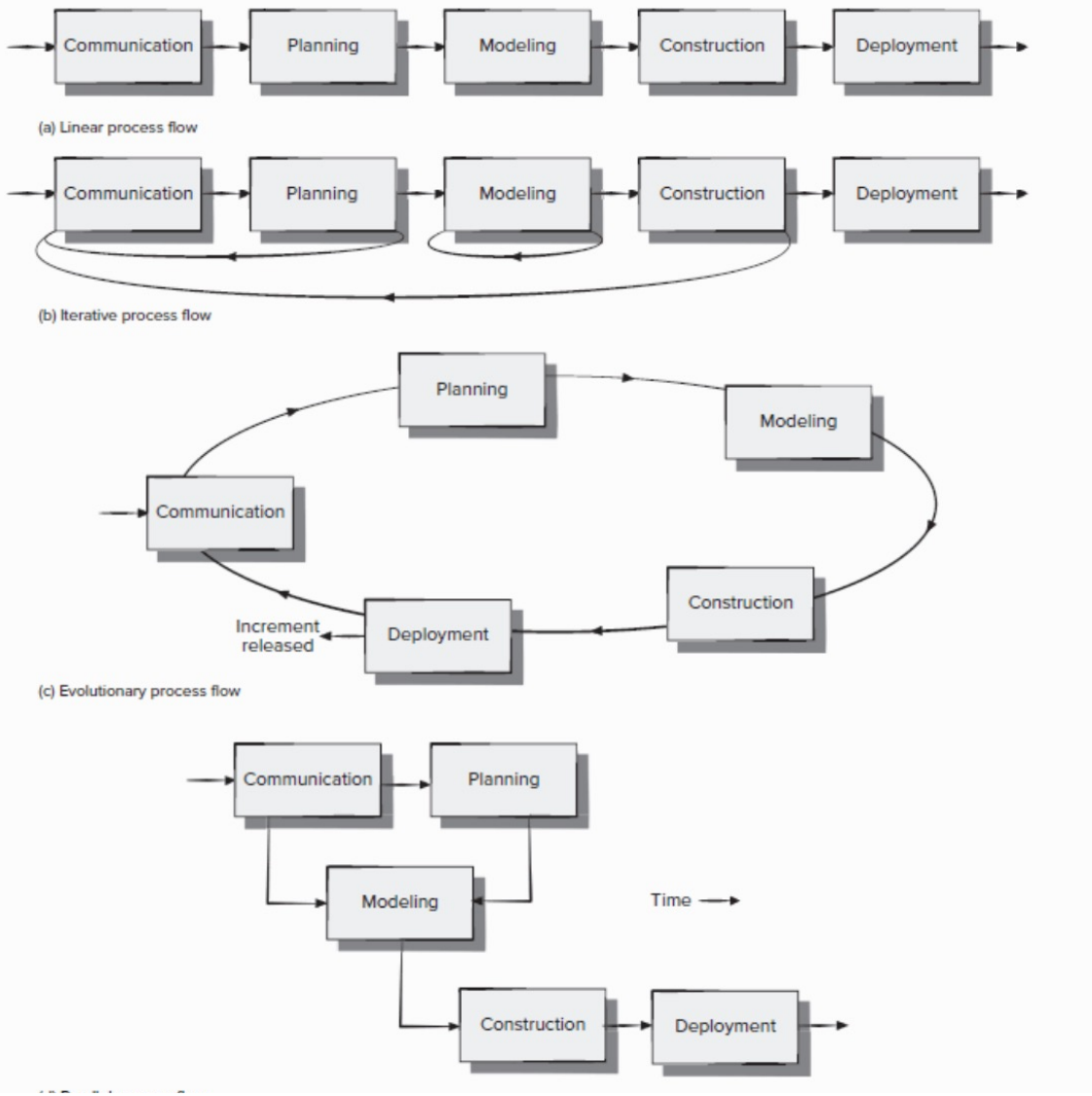
Software Process Model

- A generic process framework for software engineering defines five framework activities—communication, planning, modeling, construction, and deployment.
- In addition, a set of umbrella activities—project tracking and control, risk management, quality assurance, configuration management, technical reviews, and others—are applied throughout the process



Process Flow

- Process flow—describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time.



Software Process Model

- **Traditional Models**
  - Waterfall model
  - Iterative Model
  - Spiral Model

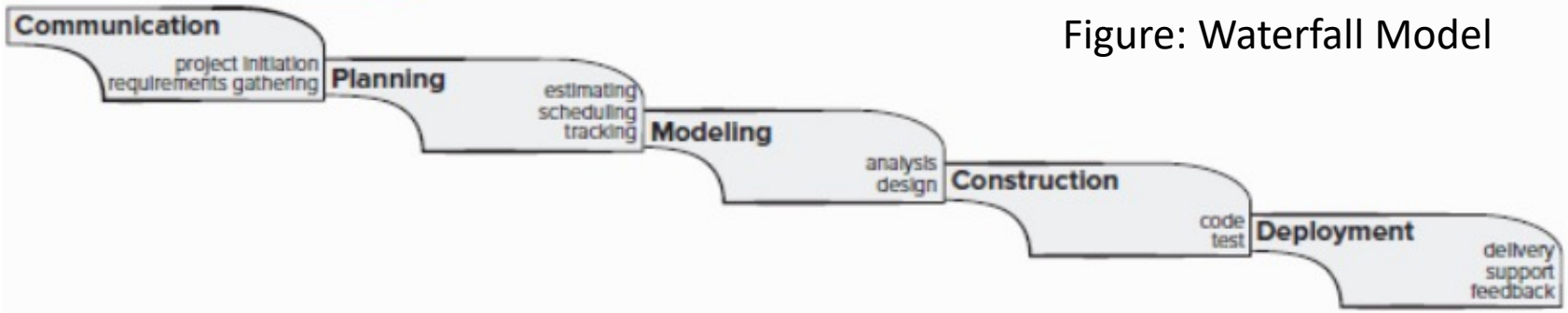


Figure: Waterfall Model

- The **waterfall model**, sometimes called the linear sequential model, suggests a systematic, sequential approach3 to software development that begins with customer specification of requirements and progresses through planning, modeling, construction, and deployment, culminating in ongoing support of the completed software

Software Process Model

- **Traditional Models**
  - Waterfall model
  - Iterative Model
  - Spiral Model

- In the **Iterative and Incremental** software development model, the process begins with implementing a few requirements in a single or several iterations or increments. In later iterations, the software is expanded with additional and more complex features until the expected system is created. At the end of each iteration, the development team has a new software version that gets increasingly advanced. This approach allows the team to review the product at the end of every increment, identify its shortcomings, and implement corresponding changes to improve it.

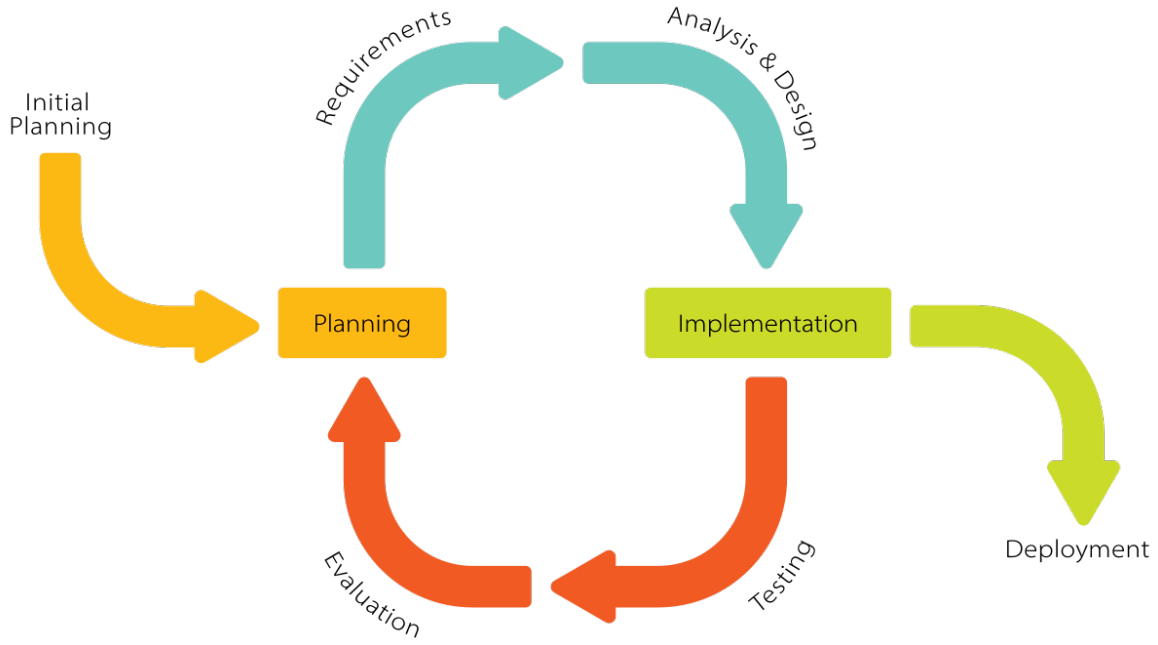


Figure: Iterative Model

Software Process Model

- **Traditional Models**
  - Waterfall model
  - Iterative Model
  - Spiral Model

- Originally proposed by Barry Boehm [Boe88], the spiral model is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model.

- Using the spiral model, software is developed in a series of evolutionary releases. During early iterations, the release might be a model or prototype. During later iterations, increasingly more complete versions of the engineered system are produced.

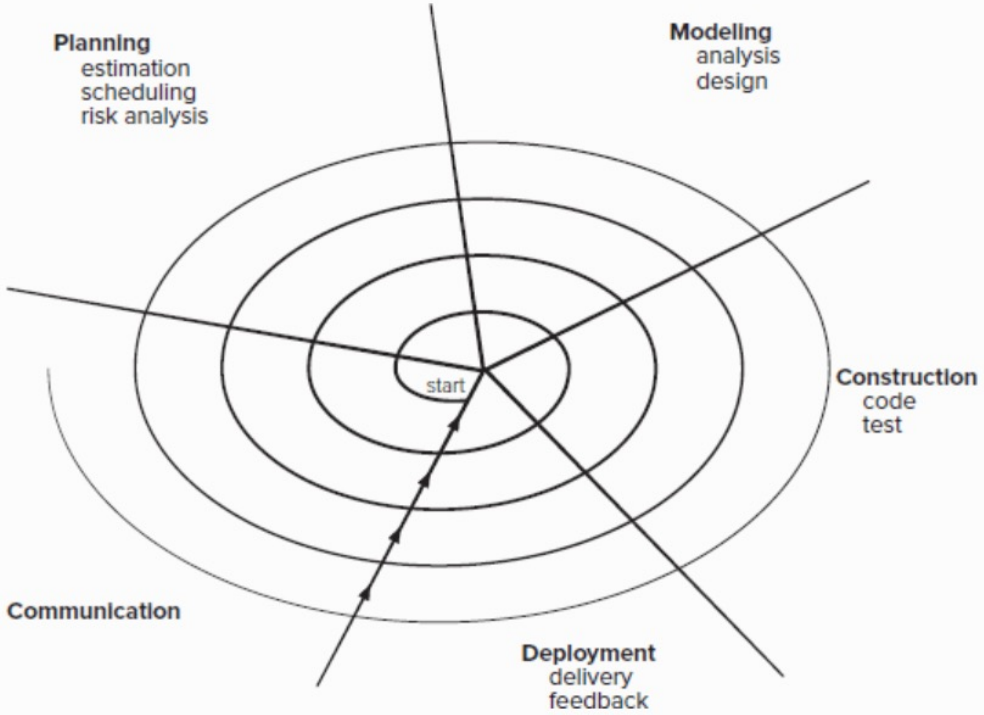
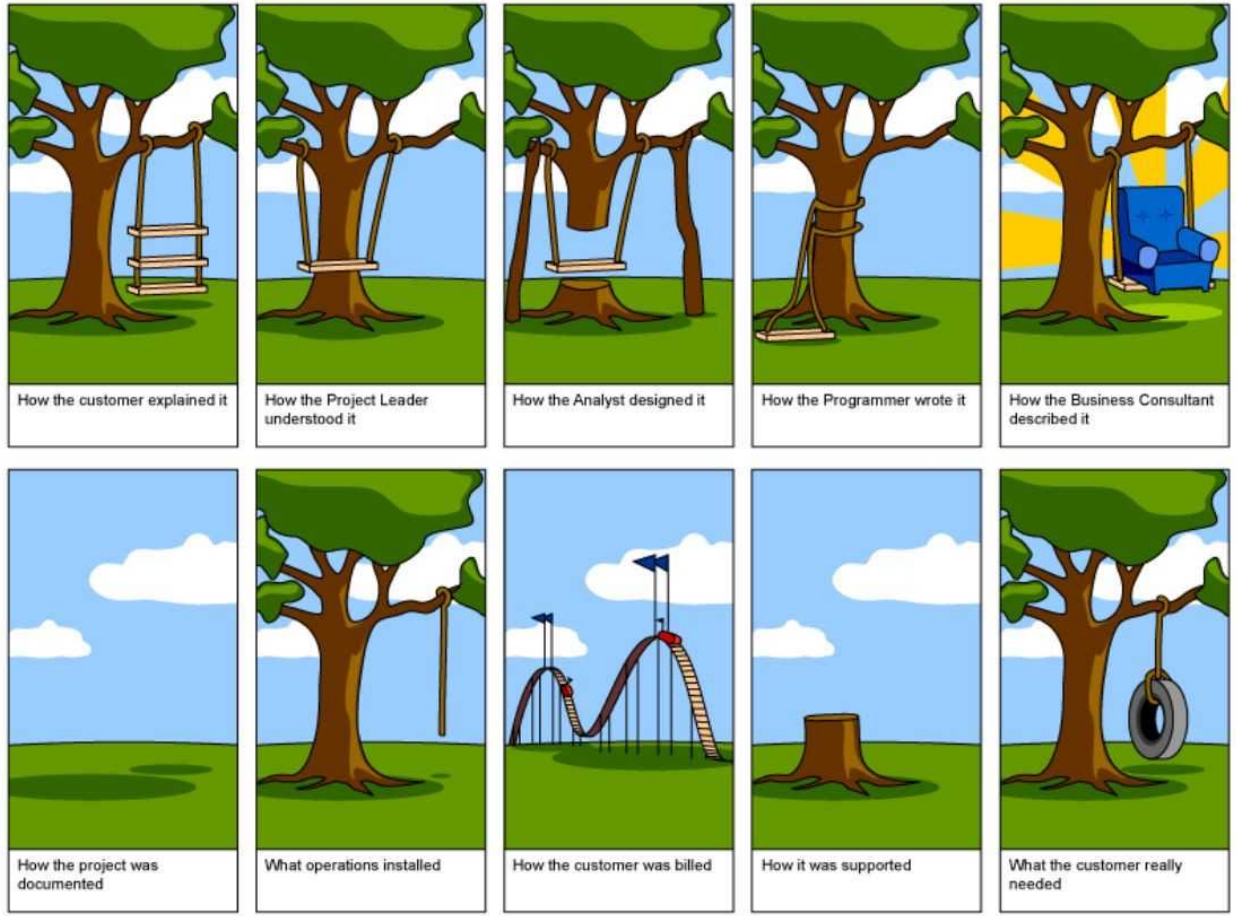


Figure: Iterative Model

Software Process Model

- **Agile Models**

- Customer priorities will change as the project proceeds.
- For many types of software, design and construction are interleaved. That is, both activities should be performed in tandem so that design models are proven as they are created. It is difficult to predict how much design is necessary before construction is used to prove the design.
- Analysis, design, construction, and testing are not as predictable (from a planning point of view) as we might like.
- "How do we create a process that can manage unpredictability?



Software Process Model

- **Agile Models**
  - Agile software engineering combines a **philosophy** and **a set of development guidelines**.
  - The **philosophy** encourages customer satisfaction and early incremental delivery of software; small, highly motivated project teams; informal methods; minimal software engineering work products; and overall development simplicity.
  - The **development guidelines** stress delivery over analysis and design (although these activities are not discouraged)

Criteria	Waterfall Model	Agile Model	Spiral Model	V-Model	Incremental Model	Prototype Model	DevOps Model
Description	Sequential model with distinct phases.	Iterative, incremental development with flexibility.	Iterative model emphasizing risk management.	Enhanced Waterfall with testing at each stage.	Delivers functionality in increments.	Develops a prototype to refine requirements.	Combines development and operations for continuous delivery.
Best Use Case	Well-defined, stable requirements.	Projects with evolving requirements and frequent changes.	High-risk, complex projects requiring frequent evaluation.	Projects requiring rigorous validation and quality control.	Projects requiring gradual functionality delivery.	Unclear or ambiguous requirements.	Projects needing continuous integration and frequent delivery.
Flexibility	Low	High	Moderate	Low	Moderate	High	High
Customer Involvement	Limited to early phases.	High; continuous feedback and interaction.	Moderate; involved during reviews.	Low; involved in the beginning.	Moderate; involved for each increment.	High; feedback on prototypes.	High; feedback during deployments.
Risk Management	Low	Moderate	High; focuses on risk mitigation.	Low	Moderate	Moderate	High
Delivery	At the end of all phases.	Frequent delivery through sprints.	Cyclical delivery in stages.	Final delivery after validation.	Delivered incrementally.	Delivered incrementally.	Continuous delivery of updates.

Criteria	Waterfall Model	Agile Model	Spiral Model	V-Model	Incremental Model	Prototype Model	DevOps Model
Time Efficiency	Slow due to sequential phases.	Faster due to iterative development.	Slower; cycles add time but reduce risk.	Moderate due to validation phases.	Faster for initial delivery but depends on increments.	Faster to clarify requirements.	Fast; focuses on automation and continuous updates.
Cost Efficiency	Predictable costs for stable projects.	Costs may increase with frequent changes.	High initial costs for risk management.	Predictable but rigid costs.	Moderate; incremental effort affects costs.	Moderate to high for building prototypes.	High for initial automation setup but reduces long-term costs.
Quality Control	Low; testing occurs late.	High; iterative testing improves quality.	High; continuous evaluation and testing.	High; emphasis on validation.	Moderate; quality improves over increments.	Moderate; quality depends on prototype feedback.	High; ensures high quality through CI/CD pipelines.
Scalability	Low; difficult to scale.	High; supports modular and evolving requirements.	High; accommodates changes and risks.	Low; rigid structure limits scaling.	High; can adapt to new increments.	Low; limited to the prototype scope.	High; supports scaling through automation and modular architecture.
Examples	Banking software, government projects.	Social media apps, startups.	Aerospace, defense systems.	Medical software, safety-critical systems.	E-commerce platforms, mobile apps.	Proof-of-concept projects, UI-heavy apps.	SaaS platforms, cloud services.

Assignment 1

- Please do it in a group of two students
- The deadline for the submission of the assignment is Wednesday, December 11, 2024 23:59.
- send in pdf along with a cover page
- Plagiarism will be checked and it is Plagiarism is considered as serious academic dishonesty.

The Exam Control Office of Pokhara University aims to digitize its services, making all required tasks accessible online. The standard operating procedures for each task are already well-defined. While the project has a reasonable timeline for completion, it must be executed within a limited budget.

Which software process model would be most suitable for developing this software? Analyze and discuss the advantages and disadvantages of the chosen process model

Assignment-2

- A **Flight Control Software System** is a specialized, safety-critical software designed to manage and regulate an aircraft's flight control surfaces, such as ailerons, rudders, elevators, and flaps. Its primary purpose is to ensure stability, maneuverability, and safety during all phases of flight, including takeoff, cruising, and landing.
- **Key Functions:**
  1. **Automatic Flight Control**: Supports autopilot operations to maintain altitude, heading, and speed.
  2. **Manual Flight Assistance**: Enhances pilot control by interpreting input from the cockpit controls and translating them into precise movements of the flight surfaces.
  3. **Flight Envelope Protection**: Ensures the aircraft operates within safe limits by preventing dangerous maneuvers, such as stalling or exceeding structural stress thresholds.
  4. **Sensor Integration**: Gathers data from sensors (e.g., airspeed, altitude, and gyroscopic data) to dynamically adjust control actions.
  5. **Redundancy and Fault Tolerance**: Incorporates multiple levels of redundancy to continue functioning in case of component or system failure.
- **Importance:**
  - Flight control software systems are critical for modern fly-by-wire aircraft, where traditional mechanical and hydraulic controls are replaced or augmented by electronic systems. Their performance and reliability are essential to ensure passenger safety and compliance with stringent aviation standards like DO-178C (Software Considerations in Airborne Systems and Equipment Certification).

Which software process model would be most suitable for developing this software? Analyze and discuss the strengths and weaknesses of the chosen process model

Assignment-3

Imagine developing a smart tourist guide system that provides end-to-end services for tourists. This will be a public application accessible to all users. The goal is to create engaging, value-added features while building a community and ensuring sustainability through revenue generation. However, the specific features that will attract and benefit users are yet to be identified.

Which software process model would be most suitable for developing this software? Analyze and discuss the strengths and weaknesses of the chosen process model.

Introduction to Requirement Engineering

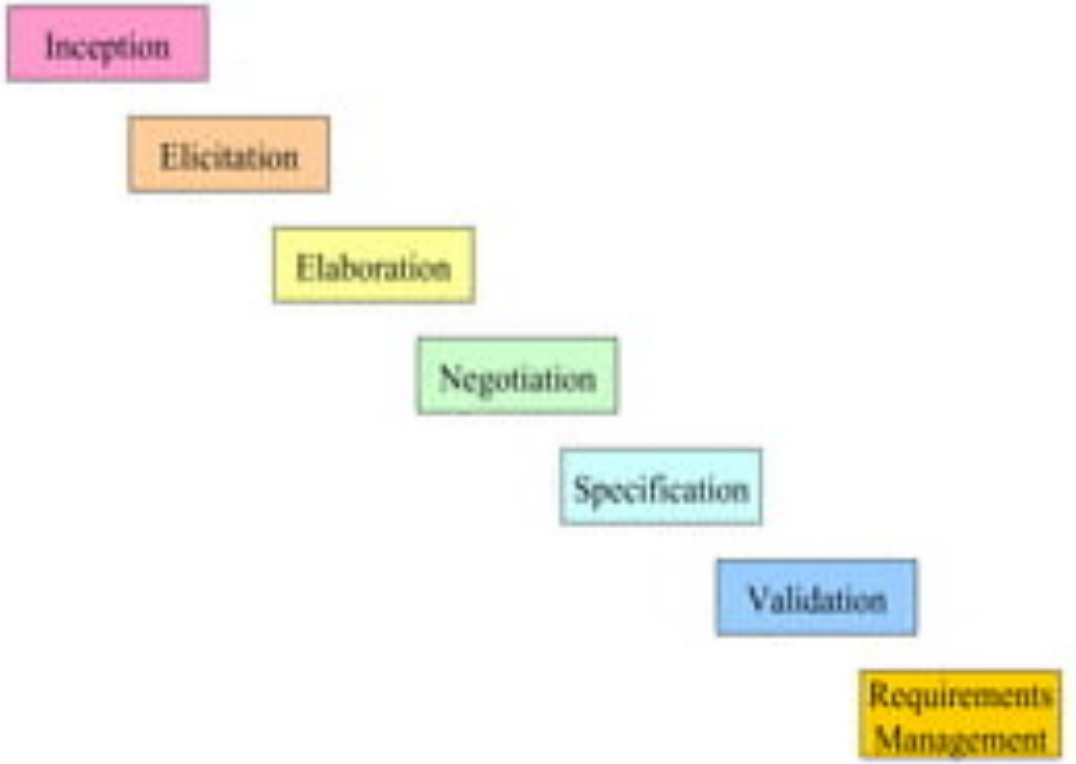
Introduction to Requirement Engineering

- “A customer walks into your office, sits down, looks you straight in the eye, and says, “I know you think you understand what I said, but what you don’t understand is what I said is not what I meant.” Invariably, this happens late in the project, after deadline commitments have been made, reputations are on the line, and serious money is at stake.”[1]

- [1]Excerpt From: PRESSMAN;. “ISE EBook Online Access for Software Engineering: A Practitioner's Approach.” Apple Books.

Introduction to Requirement Engineering

- From a software process perspective, requirements engineering is a major software engineering action that begins during the communication activity and continues into the modeling activity.
- Requirements engineering builds a bridge to design and construction.
- Requirements engineering encompasses seven tasks with sometimes muddy boundaries: inception, elicitation, elaboration, negotiation, specification, validation, and management.



Introduction to Requirement Engineering

- Inception
  - At project inception, you establish a basic understanding of the problem, the people who want a solution, and the nature of the solution that is desired.

- Elicitation
  - Ask the customer, the users, and others what the objectives for the system or product are, what is to be accomplished, how the system or product fits into the needs of the business, and finally, how the system or product is to be used on a day-to-day basis.

- Elaboration
  - Elaboration is driven by the creation and refinement of user scenarios obtained during elicitation. These scenarios describe how the end users (and other actors) will interact with the system.

Introduction to Requirement Engineering

- Negotiation
  - It's also relatively common for different customers or users to propose conflicting requirements, arguing that their version is "essential for our special needs.
  - These conflicts need to be reconciled through the process of negotiation.
- Specification
  - A specification can be a written document, a set of graphical models, a formal mathematical model, a collection of usage scenarios, a prototype, or any combination of these.
  - Downloda the template from: https://web.cs.dal.ca/~hawkey/3130/srs_template-ieee.doc
- Validation
  - Requirements validation examines the specification to ensure that all software requirements have been stated unambiguously; that inconsistencies, omissions, and errors have been detected and corrected; and that the work
  - "The primary requirements validation mechanism is the technical review"
- Requirement management:
  - Requirements management is a set of activities that help the project team identify, control, and track requirements and changes to requirements at any time as the project proceeds.

Introduction to Requirement Engineering

- *Consider developing a Know Your Customer (KYC) software specifically designed for fintech companies. This system would operate in a cloud environment and be offered to clients as a Software-as-a-Service (SaaS) solution. The primary users of the application would be the customers or potential customers of large banks and financial institutions.*
- *What can be the non-functional requirements for the above case.*

Introduction to Requirement Engineering

- A nonfunctional requirement (NFR) can be described as a quality attribute, a performance attribute, a security attribute, or a general constraint on a system.
- These are often not easy for stakeholders to articulate.

Class Discussion

Imagine you are designing a mobile application for online banking targeted at senior citizens. Prioritize the non-functional requirements for the application.

Security, Performance, Reliability, User friendliness, Scalability, compliance, Support

Introduction to Requirement Engineering

- Homework
  - Write an essay on importance of Requirements Engineering.